Embeddings

Turning Data into Meaningful Vectors

---

1. What an Embedding Is

An embedding is a mapping from a discrete or high-dimensional object - a word, sentence, image, user, or item - to a dense vector of real numbers in a lower-dimensional continuous space. The goal is for geometric relationships in that space (distance, direction) to reflect semantic or functional relationships in the original data.

2. Why Not One-Hot Vectors?

One-hot encoding represents each item as a sparse vector with a single 1 and the rest 0s. It wastes space, has no notion of similarity (every pair of distinct items is equally 'far apart'), and scales poorly with vocabulary size. Embeddings fix this by learning compact, dense representations where similar items end up close together.

3. How Embeddings Are Learned

- **Prediction-based (word2vec, GloVe)**: train a shallow model to predict a word from its context (or vice versa); the learned weight matrix becomes the embedding table.
- **Contrastive learning**: pull representations of similar/paired items closer and push dissimilar ones apart (e.g., CLIP for image-text, sentence-transformers for text).
- **As a byproduct of a larger task**: the hidden layers of a trained neural network (e.g., a transformer's token or sentence representations) can be reused directly as embeddings.
- **Autoencoding**: compress input into a bottleneck vector and reconstruct it, forcing the bottleneck to capture essential structure.

4. Measuring Similarity

Once items live in vector space, similarity is computed with a distance or similarity metric:

```
cosine_similarity(a, b) = (a . b) / (|a| * |b|) euclidean_distance(a, b) =
sqrt( sum_i (a_i - b_i)^2 )
```

Note: Cosine similarity is popular for text embeddings because it ignores magnitude and focuses on direction.

5. Famous Analogy Property

Well-trained word embeddings capture linear relationships, famously:

```
vector('king') - vector('man') + vector('woman') ~= vector('queen')
```

This shows embeddings encode not just similarity but relational structure.

6. Types of Embeddings

- **Word embeddings**: word2vec, GloVe, fastText - one vector per word, context-independent.
- **Contextual embeddings**: BERT, GPT-style models - same word gets a different vector depending on surrounding context.
- **Sentence/document embeddings**: a single vector summarizing an entire passage, used heavily in semantic search and retrieval-augmented generation.
- **Image embeddings**: outputs of a vision backbone (CNN or ViT) that summarize visual content.
- **Multimodal embeddings**: shared vector space across text, images, audio (e.g., CLIP) so items of different modalities can be compared directly.
- **Graph/entity embeddings**: represent nodes in a graph (users, products) for recommendation.

7. Embeddings in Practice: Vector Search

A common pipeline: embed a large corpus of documents offline, store the vectors in a vector database or ANN index (FAISS, HNSW, ScaNN), then embed an incoming query with the same model and retrieve the nearest stored vectors. This is the retrieval step behind semantic search and retrieval-augmented generation (RAG) systems.

8. Practical Considerations

- **Dimensionality**: higher dimensions capture more nuance but cost more storage/compute; typical text embeddings range from 256 to 3072 dimensions.
- **Normalization**: many systems L2-normalize vectors so cosine similarity reduces to a dot product.
- **Domain match**: an embedding model trained on general web text may underperform on specialized domains (legal, medical) - fine-tuning or domain-specific models often help.
- **Consistency**: query and document embeddings should come from the same model/version to be comparable in the same space.